PSAI_123

Iris

- **Team Name:** Ideal Labs
- **Payment Address:** 0xB7E92CCDE85B8Cee89a8bEA2Fd5e5EE417d985Ad (DAI)
- **Level:** 3
- **Status:** Terminated

Project Overview :page_facing_up:

This is a follow-up grant to Iris:

- https://github.com/w3f/Grants-Program/pull/812
- https://github.com/w3f/Grants-Program/blob/master/applications/iris.md

Overview

Iris is a decentralized **data exchange protocol** that enables a secure data ownership, access management, and delivery layer for Web 3.0 applications. It is infrastructure for the decentralized web. By building a cryptographically verifiable relationship between storage and ownership, Iris provides a data exchange which enables the transfer and monetization of access to and ownership of data across chains, smart contracts and participants in the network or connected through a relay chain. Iris provides security, reputation, and governance on top of storage, enabling data ownership, monetization, access management, capabilities to define unqiue business logic for data access and authorization, as well as smart contract support for content delivery. It applies defi concepts to data to represent off-chain assets in an on-chain context by representing data as a unique asset class with access to the underlying data controlled by ownership of assets minted from the asset class.

Project Details

The intention of this follow-up grant is to implement several features that enable Iris to be a secure data exchange protocol with support for multiple storage backends, capabilties for the development of unique data access and authorization models, and that lays the foundation for future governance and moderation capabilities. In our system, data owners associate their data with any number of ‘data spaces’ that each have specific rule sets and inclusion policies. We also lay the foundations for our encryption scheme (we will use a threshold encryption mechanism, though the implementation of this is out of scope for this proposal) by introducing the concept of the proxy node that are the linchpin for threshold encryption, as well as allows data owners and data consumers to run light clients (as they no longer are required to run the full node and add data to the embedded IPFS node). Additionally we introduce “composable access rules” and an associated “rule executor”, allowing data owners to define unique business logic that enables custom access and authorization models to their data. Further, we build a generic storage connector module which allows Iris the abilitiy to support multiple downstream ‘cold’ storage systems (such as using XCM to communicate with the Crust network) as well as ‘hot’ storage via IPFS, supported by proxy nodes, that facilitates the transfer of data between them in an offchain context. Lastly, we will build a javascript SDK to allow user interfaces for dapps built on Iris to easily build applications and interface with Iris.

This proposal makes several improvements on top of the existing Iris blockchain, specifically in terms of security, extensibility, data organiziation, and data ingestion/ejection workflows.

To summarize, in the following we propose:

- the introduction of “data spaces”
- the implementation of “composable access rules” to apply custom business logic to data
- the introduction of “proxy nodes” that enable threshold encryption within the network
- implementation of an offchain client to send/receive data to/from proxy nodes
- introduce a configurable storage layer to support storage in multiple backends as well as usage of go-ipfs for hot storage
- a javascript SDK to allow dapp developers to easily build front ends for smart contracts on Iris

For further details, we direct the reader to view our whitepaper draft here:
https://www.idealabs.network/docs

Note: The whitepaper is outdated, specifically in sections where references to the storage system are

made.

Tech Stack

Frameworks/Libraries:

- go-ipfs
- Substrate
- Crust Network

Languages:

- Rust
- Go
- Javascript/Typescript

Terminology

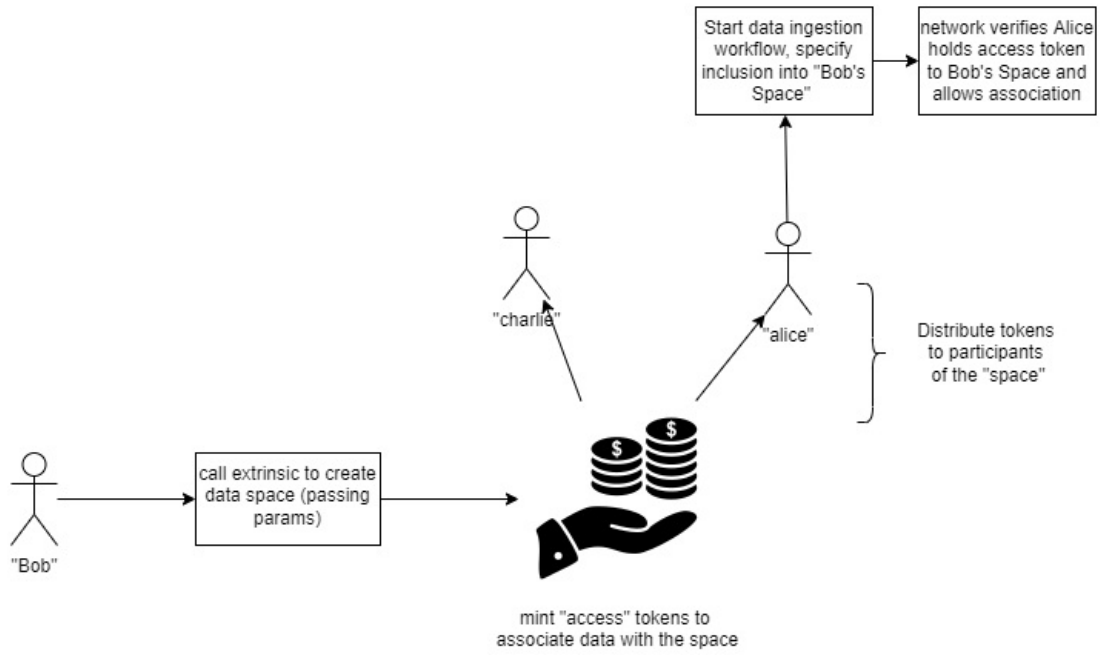
- Data Asset Class and Data Asset: A **data asset class** is an asset class which is associated with some owned data which has been ingested into the Iris storage system at some point in the past. A **data asset** is an asset minted from the data asset class.
- Data Owner: A **data owner** is any node that has initiated the ingestion of some data and owns a data asset class. In the following, we may refer to data owners as “Alice”.
- Data Consumer: A **data consumer** is any node that owns a data asset minted from a data asset class. In the following, we may refer to data consumers as “Bob”.
- Proxy Node: A **proxy node** is any node that has the minimum required hardware requirements and has staked the minimum required amount IRIS tokens to become a proxy node.

Data Spaces

A data space is a user-defined configuration and rule set that allows users to group data together into curated collections. More explicitly, the data space lays the foundation to enable user-defined moderation rules within the network and provides a means to securely associate disparate data sets with one another (i.e. under an ‘organization’). All data in Iris exists within the context of a space, however, there is no limit to the number of spaces a data owner may associate their data with.

Under the hood, the ‘data space’ is an asset class which is mapped to a set of configuration items that determine the inclusion and moderation policies of the space. These asset classes exist much the same as the concept of ‘data assets’ from the original Iris proposal. In a data space with a private inclusion policy, only nodes holding tokens minted from the data space asset class are authorized to associate data with the data space. We see this as an additional market opportunity for the data economy, as ownership of a space may become important based on the sum of data stored within. Data spaces may also be subject to composable access rules at some point in the future.

Further, data spaces form the basis for moderation, or curation, within the network. Each data space may have a set of rules associated with it which limits not only the type of data that can be associated with the space, but also with the contents of the data. We intend to accomplish this through the execution of machine learning models, bayesian filters, and more, within a trusted execution environment. However, that work is outside the scope of this proposal.



data spaces diagram

Composable Access Rules and Data Access Authentication via a Rule Executor

Iris enables data owners to determine rules that consumers must follow when requesting their data from the network which, when met, should authorize the caller to access the data associated with the asset class. These rules could include: limited use access tokens, minimum token requirements, time sensitive tokens, and many other use cases.

We accomplish this through a set of smart contracts called composable access rules (CARs) and an associated 'rule executor' contract. The rule contracts are composable in the sense that the data owner may specify any number of rules, each of which must be validated when a consumer requests data. In general, each rule stipulates a condition which, when met, results in the consumer being blocked from accessing data. For example, a "limited use token contract" would track the number of times a token is used to fetch data, and forces that token to be burned once the threshold is reached. If at end of execution of each CAR no false result has been returned, then the consumer can proceed to fetch the data. In general, the greater the number of CARs, the more costly it will be for consumers to access the data, since they must pay for each one to be executed.

In order to associate CARs with an asset class, each CAR must be executed as part of a 'rule executor' contract which makes cross-contract calls to each rule specified. After execution of each rule, the executor submits the results on chain. In general, we assume that the result submission is a boolean value, where true implies each rule evaluated to true and data access is authorized, and false implies some rule failed and data is not accessible. When an executor contract's address is associated with the on-chain asset class, Iris uses a mechanism to 'unlock' the data for the authorized address for a limited period of time (until it is fetched for the first time). To this end, we introduce a new pallet, `iris-ejection`, which handles the management of this locking mechanism, submission of rule executor results, and association of rule executor contract addresses to asset classes.

A composable access rule must implement the following functions, which is enforced via a trait:

- an `execute(asset_id, consumer_address)` function that produces a boolean output. This function accepts the asset id of an asset class in Iris and a consumer address. It encapsulates the 'access rule' logic.

Proxy Nodes and offchain clients for data ingestion/ejection

Proxy nodes form the basis of secure data ingestion and ejection from the network. Ultimately, these nodes will be responsible for powering Iris's threshold encryption mechanism. This mechanism is out of the scope of the current proposal, however, we will create this node role to provide the foundation for subsequent work in which the threshold encryption scheme will be implemented, as well as make data ingestion/ejection workflows easier for data owners and consumers. In the implementation proposed here, these nodes simply act as ingress and egress points to/from the IPFS network. Though proxy nodes do not increase the security of the network at this point, they do remove the dependency on data consumers and data owner to run a full Iris node to enable data ingestion and ejection.

Putting data spaces and proxy nodes together, we arrive at the following design:



proxy nodes

Offchain Client

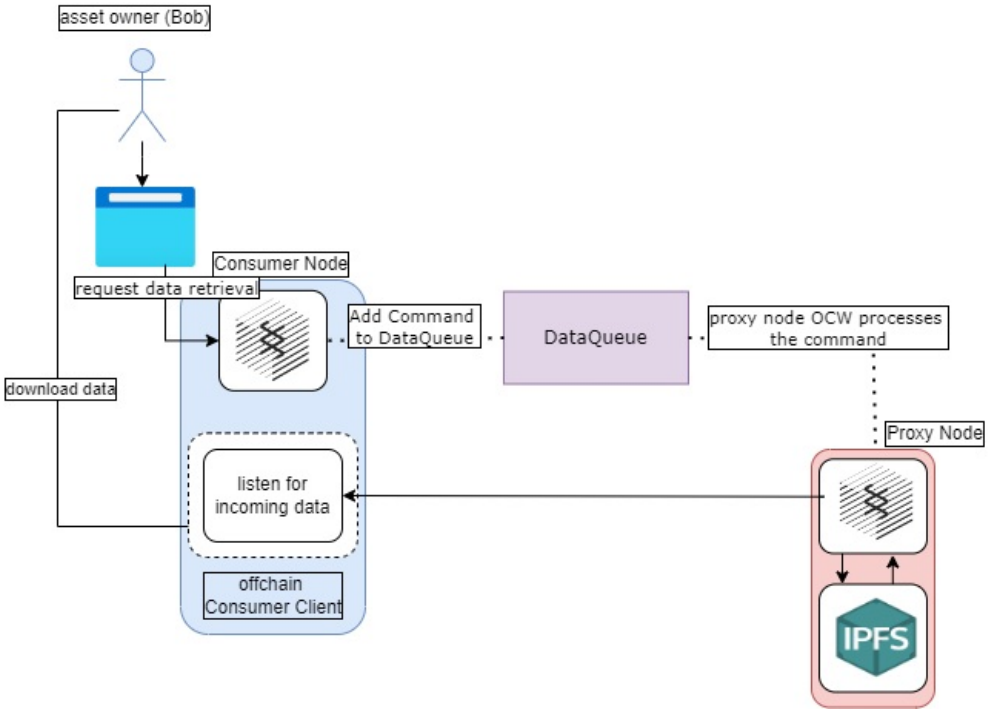
The inclusion of proxy nodes impacts data ingestion and ejection workflows. We will build a secure offchain client using Go in order to serve files for ingestion by proxy nodes and to listen for data sent by proxy nodes when requested from the network.

In the initial design of Iris, data ingestion functioned by allowing a data owner, who is running a full iris node, to add some data to their embedded IPFS node to gossip with a validator node, who would then add the data to their own embedded IPFSS node, which is connected with other validator nodes. This poses several issues. Not only is it insecure, but also it forces data owners to always run a full node which limits the ease of use of the system. Similarly, data ejection functioned by allowing a data consumer to directly connect their embedded IPFS node to a validator node to retrieve data from the 'validator network', which introduced a similar set of issues that data owners would face. Proxy nodes allow us to circumvent both of these issues by acting as a gateway to the IPFS network. To accomplish data ingestion, nodes will run an offchain client, which acts as a file server that only authorized proxy nodes can call into.



data ingestion

For data ejection, data consumers run the offchain client which will listen for connections from proxy nodes, accept authorized connections, and provide data consumers the ability to fetch data without running a full node.



data ejection

Encryption/Decryption

As part of this proposal, we will implement a semi-centralized approach to securing data. There are significant security issues with the design we are proposing that we are aware, specifically when any authority node chooses to act maliciously. As we are remaining a proof of authority network for the time being, we are choosing to forgo threshold encryption while we continue to build the infrastructure needed to support it.

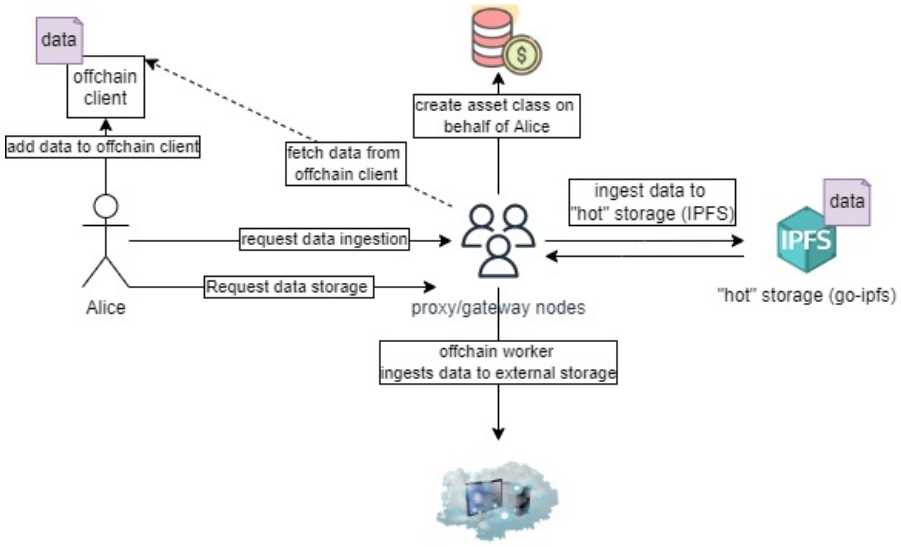
Our approach to encryption relies on a semi-static set of authority nodes. We propose a system where a data owner's offchain client encrypyts their data so it may be decrypted by some specific proxy node who has been selected to handle the ingestion of their data into the network. Once this is done, the chosen proxy node decrypts and re-encrypts the data so it can be decrypted by any current proxy node in the network. When data is requested from the network, a proxy node then fetches the data from the 'hot' storage system (discussed here), decrypts it, re-encrypts it for the caller, and delivers it to the offchain client. This encryption and decryption happens offchain, but not in a TEE, which we are aware poses increased security risks which we will remediate once we implement threshold encryption.

Proxy Node Reward Structure

Each data ingestion and ejection transaction has an additional transaction fee which is then pooled together and distributed to proxy nodes at the end of a session based on their participation during the session. This will function similarly to how proof of stake consensus algorithms reward validator nodes when new blocks are accepted into the chain.

Storage System Integration

There are two storage layers in Iris, a ‘hot’ storage layer which is supported by the proxy nodes, and a ‘cold’ storage layer which exists offchain.



hot-cold-storage

We will build a generic pallet that allows for any given storage backend to be configured for use with Iris. The intention behind this is that it may allow the network to function agnostically of any one given storage solution. The pallet will expose two main extrinsics, a ‘read’ extrinsic and a ‘write’ extrinsic, which send commands to proxy nodes to either ingest data from a configured storage system into hot storage, or to store data available in hot storage into cold storage. This approach allows us to support multiple storage backends, as well as provides us the freedom to implement our own storage system in the future without impacting the user experience.

Initally, we will demonstrate how this can be used to allow connections to an external storage backend by adapting the connector to an external, centralized data store (e.g. either a locally running file server or AWS S3), which may be useful in B2B/C solutions who want to maintain ownership of their own storage system.

To realize a decentralized solution to storage, we will use the same approach to implement a pallet which acts as a connector to the Crust network via XCMP. Explicitly, we intend to use the xStorage pallet provided by the crust project, which allows for us to place storage orders on the Crust network. Crust IPFS nodes will then fetch data, as based on the given CID and Multiaddress, from an Iris proxy node. Further, we will integrate with the xTokens pallet to enable the usage of our native token (IRIS) to pay for storage fees. The interactions with Crust (and this would be the same for any given storage system) will be abstracted away from the user through the generic storage connector pallet, which exposes read/write functionality. We intend to follow the approach described here as well as open a dialogue with the Crust team in order to ensure all testing is sufficient and successful. We enable proxy nodes to ingest data from cold storage by reading from the IPFS gateway in which Crust has stored the data.

In order to properly test and verify this, we will also setup our own relay chain with both Iris and Crust deployed as parachains.

IRIS SDK

We intend to adapt the user interface developed as part of the initial Iris grant proposal for usage as an SDK for dapp developers on Iris. The SDK will facilitate development of user interfaces for contracts running on the Iris blockchain. As an example, the Iris Asset Exchange UI from the initial grant proposal is one such application. We will adapt the current Iris UI to use this new SDK as well as demonstrate how to develop a simple data marketplace.

Specifically, we intend to adapt the SDK from the functions available in these files: https://github.com/ideal-lab5/ui/tree/master/src/services.

Prior Work

This proposal is a followup to a completed web3 foundation grant which laid the foundation for these enhancements. With our initial proposal, we delivered:

- a layer on substrate that builds a cryptographic relationship between on-chain asset ownership and actual off-chain data storage
- a naïve validator-based storage system that encourages storage of 'valuable' data
- support for contracts and a rudimentary decentralized exchange to purchase and sell data access rights

In addition to these items, the Iris fork of substrate has been upgraded to by in sync with the latest substrate master branch and we have also upgraded the runtime to use the latest version of rust-ipfs (see here: d9e14294c89abd2c085e91d8312041245b5d3b35).

Limitations and Expectations

Iris is *not* intended to act as a decentralized replacement for traditional cloud storage or to be a competitor to existing data storage solutions. In fact, Iris is not really a storage layer at all. Further, this proposal does not address security (i.e. threshold encryption) in the network, nor does it appproach moderation capabilities, though it does lay the foundation for those items to be implemented in the next phase of development.

Ecosystem Fit

- Where and how does your project fit into the ecosystem?
    - Iris intends to be infrastructure for dapps that leverage decentralized storage and ownership capabilities. Further, our long term intention is to participate in a parachain auction once Iris is feature complete, after which we will be able to provide cross-chain data ownership that is cryptographically linked with their stored data.
- Who is your target audience (parachain/dapp/wallet/UI developers, designers, your own user base, some dapp's userbase, yourself)?
    - The target audience is very wide ranging as we aim to provide infrastructure for dapps and potentially other parachains to allow them to easily take advantage of decentralized storage capabilities. We also aim to provide tools for data owners and content creators to share or sell their data without a middleman, determining their own prices and business models. Additionally, data spaces allow organizations to control which data can be associated with them, which may allow Iris to be a used in B2B or middleware within existing centralized applications.
- What need(s) does your project meet?
    - The basis of Iris is the creation of a cryptographically verifiable relationship between data ownership, accessibility, and storage. We aim to treat data as an on-chain asset as well as provide a robust governance and moderation framework, which can assist in IP protections and safeguarding the network from being used for abusive or malicious purposes (such as hosting malware, illicit or illegal materials, plagiarized materials, etc.).
- Are there any other projects similar to yours in the Substrate / Polkadot / Kusama ecosystem?

Team :busts_in_silhouette:

Team members

- Tony Riemer: co-founder/engineer
- Tom Richard: engineer
- Sebastian Spitzer: co-founder
- Brian Thamm: co-founder

Contact

- **Contact Name:** Tony Riemer
- **Contact Email:** driemworks@idealabs.network
- **Website:** https://idealabs.network

Legal Structure

- **Registered Address:** TBD, in progress
- **Registered Legal Entity:** Ideal Labs (TBD)

Team’s experience

Ideal Labs is composed of a group of individuals coming from diverse backgrounds across tech, business, and marketing, who all share a collective goal of realizing a decentralized storage layer that can facilitate the emergence of dapps that take advantage of decentralized storage.

Tony Riemer

Tony is a full-stack engineer with over 6 years of experience building enterprise applications for fortune 500 companies, including both Fannie Mae and Capital One. Notably, he was the lead engineer of Capital One’s “Bank Case Management” system, has lead several development teams to quickly and successfully bring products to market while adhering to the strictest code quality and testing practices, and has acted as a mentor to other developers. Additionally, he holds a breadth of knowledge in many languages and programming paradigms and has built several open-source projects, including a proof of work blockchain written in Go, an OpenCV-based augmented reality application for Android, as well as experiments in home automation and machine learning. More recently, he is the founder and lead engineer at Ideal Labs, where he single-handedly designed the Iris blockchain and implemented the prototype, which was funded via a web3 foundation grant. He is passionate about decentralized technology and strives to make the promises of decentralization a reality.

Tom Richard

A programming fanatic with six years of professional experience who always thrives to work on emerging technologies, especially in substrate and cosmos to create an impact in the web 3.0 revolution. I believe WASM is the future and has great leverage over EVM, and that is the reason why I started building the Polkadot ecosystem, currently developing smart contract applications and tools with Rust.

Sebastian Spitzer

Sebastian started his career in marketing, from where he ventured into innovation management and built up innovation units and startup ecosystems in 3 verticals.

Essentially working as an intrapreneur in the automotive sector for the last 6 years, he managed the entire funnel from ideation to piloting and ultimately scaling digital solutions. In the last 3 years he scaled up 2 projects, adding 2m+ of annual turnover within the first 2 years to market.

In 2016 he first came into contact with the Blockchain space when scouting new tech and got deeply immersed when he saw the disruptive potentials. Since then he has become an advisor for two projects, supporting them in their growth to 200m+ market caps, worked as a dealflow manager/analyst for a crypto VC and helped founders on their early stage journey in the space.

In 2019 he authored a research paper on why the data economy in the automotive industry fails. Since then he was looking for an adequate solution and co-founded Ideal Labs 2 years later after discovering Tony’s initial work.

Brian Thamm

Brian Thamm has over a decade of experience leading large data initiatives in both the Commercial and U.S. Federal Sectors.

After several years working with companies ranging from the Fortune 500 to startups, Brian decided to focus on his passion for enabling individuals to use data to improve their decision-making by starting Sophinea. Since its founding in late 2018, Sophinea has lead multi-national Data Analytics Modernization efforts in support of the United State’s Department of State and the National Institutes of Health.

Brian believes that data-driven success is something that often requires top level support, but is only achieved when users of the information are significantly engaged and are able to seamlessly translate data into decision-making. This requires expertise, but often also novel datasets. Brian believes that blockchain ecosystems represent a generational opportunity to develop these novel data sets by providing a fairer incentive structure to reward data providers for their efforts and willingness to share.

Team Code Repos

- https://github.com/ideal-lab5/substrate
- https://github.com/ideal-lab5/iris
- https://github.com/ideal-lab5/contracts

- https://github.com/ideal-lab5/ui

Please also provide the GitHub accounts of all team members. If they contain no activity, references to projects hosted elsewhere or live are also fine.

- https://github.com/driemworks
- https://github.com/crytodevkj
- https://github.com/bgt

Team LinkedIn Profiles (if available)

- https://www.linkedin.com/in/tony-riemer/
- linkedin.com/in/haulerkonj
- https://www.linkedin.com/in/brianthamm/
- https://www.linkedin.com/in/sebastian-s-253502159/

Development Status :open_book:

As stated earlier in the prior work section, we have delivered the three milestones detailed as part of the intiial Iris grant proposal. Additionally, we have also synced the Iris repository with the latest substrate master branch as well as have upgraded to use the latest rust-ipfs version (see here: d9e14294c89abd2c085e91d8312041245b5d3b35).

Development Roadmap :nut_and_bolt:

Note: There are several items we aim to accomplish during the lifetime of this proposal that are not explicitly mentioned as part of the below milestones, including:

- properly weighting and benchmarking extrinsics
- developing automated gates for test coverage and code quality
- developing a simple testnet and CICD pipeline to automatically update the node runtimes
- updating our whitepaper with most recent results and findings
- continuing research on threshold encryption and how we can most efficiently implement it, which will also allow Iris to transition from PoA consensus to PoS consensus.

Overview

- **Total Estimated Duration:** 9 months
- **Full-Time Equivalent (FTE):** 2.5 FTE
- **Total Costs:** 47,000

Milestone 1 — Implement Data Spaces and Composable Access Rules

- **Estimated duration:** 1 month
- **FTE:** 1.5
- **Costs:** 10,000 USD

This milestone delivers two distinct deliverables.

1. We abandon the integration with rust-ipfs which was used as part of the original iris proposal. There are several reasons to do this, but the main reason is that rust-ipfs is still under development and is not yet feature complete, which severely limits its capabilities and security. Additionally, integration with rust-ipfs by embedding it within the substrate runtime significantly increases development and maintenance overhead while the benefits do not outweight usage of an external ipfs instance with go-ipfs and http bindings/communication via offchain workers. As such, this milestone will introduce breaking changes to the data ejection and ingestion workflows which are addressed in milestone 2. The results of this change implies that data ingestion and ejection are temporarily non-functional, though the underlying mechanisms are untouched. That is, this milestone introduces mechanisms to organize asset classes using dataspaces and authorization mechanisms for accessing offchain data associated with asset classes, without actually delivering the offchain data.

2. We introduce data spaces, the ability to manage data spaces associated with data, and lay the groundwork for future moderation capabilities.

3. We deliver composable access rules, the rule executor, and the iris-ejection pallet to the Iris network, allowing data owners to specify unique business models that consumers must adhere to across any number of data spaces. Specifically, we will implement a composable access rule and

associated rule executor contract which allows a token to be ‘redeemed’ only a limited number of times (e.g. ownership of a token implies you can fetch associated data only N times from the network).

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide both **inline documentation** of the code and a basic **tutorial** that explains how a user can (for example) spin up one of our Substrate nodes and send test transactions, which will show how the new functionality works.
0c.	Testing Guide	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests. We will provide a demo video and a manual testing guide, including environment setup instructions.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
0e.	Article	We will publish a medium article explaining what was achieved as part of the grant, as well as any additional learnings that we deem important throughout development. https://medium.com/ideal-labs
1.	Separate Iris node from substrate runtime fork	We migrate the existing pallets used in iris to a new repository based on the substrate node template
2.	Substrate module: DataSpaces	Create a pallet, similar to iris-assets pallet, that acts as a wrapper around the assets pallet and allows nodes to construct new data spaces and mint data space access tokens.
3.	Substrate module: Iris-Assets	Modify the iris-assets pallet to allow nodes to specify data spaces with which to associate their data. Additionally, we implement the logic required to verify that the data owner holds tokens to access the space.
4.	Contracts	Create the trait definition for a Composable Access Rule and develop a `limited use rule`, allowing a token associated with an asset id to be used only ‘n’ times. Additionally, we develop the `RuleExecutor` contract which aggregates rules and submits the results on chain.
5.	Substrate Module: Iris-Ejection	We develop a module to handle the association of rule exectors with asset classes, the submission of results from rule executors, and management of the data access locking mechanism.
6.	Substrate Module: Iris-Session	We modify the data ejection workflow to verify data authorisation via the iris ejection pallet’s “lock”.
7.	User Interface	We update the iris-ui repository so as to keep calls to extrinsics in sync with changes to parameters.

Milestone 2 - Proxy/Gateway Nodes

- **Estimated Duration:** 5.5 months
- **FTE:** 2.5
- **Costs:** 22,000 USD

This milestone delivers the infrastructure to ingest data into the network and delegate decryption rights to authorized nodes.

1. To securely ingest data, we implement two new RPC endpoints, ‘iris_encrypt’ that allows a node to encrypt data using proxy reencryption, and ‘iris_decrypt’ .
2. We create two new node types, a ‘proxy’ node which enables the proxy reencryption mechanism and a ‘gateway’ node, which enables data ingestion and asset class creation.
3. We reintegrate with IPFS by making http calls using the offchain client. We also build a module to bridge between IPFS and Iris.
4. We modify the user interface to provide a simple way to start the ingestion process, view data about asset classes, and ultimately to decrypt and download data.

Number	Deliverable	Specification

0a.	License	Apache 2.0
0b.	Documentation	We will provide both **inline documentation** of the code and a basic **tutorial** that explains how a user can (for example) spin up one of our Substrate nodes and send test transactions, which will show how the new functionality works.
0c.	Testing Guide	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests. We will provide a demo video and a manual testing guide, including environment setup instructions.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
0e.	Article	We will publish a medium article explaining what was achieved as part of the grant, as well as any additional learnings that we deem important throughout development. https://medium.com/ideal-labs
1.	Substrate Module: Iris-Proxy	Implement mechanism to allow nodes to act as a proxy and reencrypt data when requested.
2.	Substrate Pallet: IPFS	Build a pallet that enables mechanisms to issue commands to specific nodes so that their offchain worker can interact with IPFS. We do this by associating IPFS id and Iris id.
3.	Substrate Module: Gateway	Implement mechanism to allow nodes to act as a proxy and reencrypt data when requested.
4.	Encryption Mechanism	We implement a proxy reencryption mechanism as described above.
5.	RPC: Encryption RPC	The encryption RPC endpoint allows a node to send a signed message and plaintext. After verification of the signature, the plaintext is encrypted and ciphertext is returned.
6.	RPC: Decryption RPC	The decryption RPC allows a node to send a signed message and ciphertext. After verification of the signature, if the node has been delegated decryption rights (through the 'Rule Executor' from the previous milestone), then the plaintext is returned.
7.	Testnet Setup	We deploy a functional testnet to AWS that can be connected to from an iris node by specifying a custom chain specification.
8.	User Interface	We update the iris-ui repository so as to keep calls to extrinsics in sync with changes to parameters.

Milestone 3 - Storage System

- **Estimated Duration:** 2 months
- **FTE:** 1.5
- **Costs:** 12,000 USD

1. In this milestone we implement rewards and slashes for Gateway and Proxy nodes

2. This milestone delivers a generic storage pallet that can be adapted/instantiated to communicate with any given storage system. Specifically, we will demonstrate by building two distinct implementations of this adapter, one which is capable of interacting with a centralized datastore and one which uses the Crust Network. These storage systems represent the 'cold' storage capabilities of Iris. We will also invesetigate several other 'cold' storage options, such as Arweave, Filecoin, and potentially CESS (which is currently under development and funded via the w3f grants program).

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide both **inline documentation** of the code and a basic **tutorial** that explains how a user can (for example) spin up one of our Substrate nodes and send test transactions, which will show how the new functionality works.
		Core functions will be fully covered by unit tests to ensure functionality and robustness. In the

0c.	Testing Guide	guide, we will describe how to run these tests. We will provide a demo video and a manual testing guide, including environment setup instructions.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
0e.	Article	We will publish a medium article explaining what was achieved as part of the grant, as well as any additional learnings that we deem important throughout development. https://medium.com/ideal-labs
1.	Substrate Module: Gateway	We build a mechanism for gateway nodes to require payments for processing ingestion requests based on the amount of data ingested.
2.	Substrate Module: IrisProxy	We build a mechanism for proxy nodes to require payments for
3.	Substrate Module: Generic Storage Service pallet	We build a generic pallet with read and write capabilities which can be modified to support multiple storage systems.
4.	Substrate Module: Centralized Storage System	We build a storage system connector based on (2) which can read and write data to a centralized storage system (i.e. an AWS S3 or equivalent local file server).
5.	Substrate Module: Integration with Crust via the xStorage and xTokens pallets	We use the pallet developed during part 2 to use XCMP to store data in the Crust network, based on the approach outlined here.
6.	Relay Chain and Light Client	We deploy a relay chain with Iris and Crust as parachains and ensure that XCM messages are properly relayed between chains. We also build a chain specification that can be used via substrate connect to use a light client in the browser.
7.	Benchmarking, Devops and CICD	We develop pipelines needed in order to continuously update the testnet runtime when pull requests are deployed to the main branch of the github repository. We also add additional gates to this process, such as be

Milestone 4 - iris.js Javascript SDK

- **Estimated Duration:** 1 month
- **FTE:** 1.5
- **Costs:** 3,000 USD

1. This milestone delivers iris.js, a javascript SDK designed to facilitate interactions with iris and smart contracts deployed to the iris chain (i.e. with rule executor contracts). This SDK is built with the polkadot.js library and will be adapted from the functionality of the iris-ui developed in the previous grant proposal and modified as part of this current proposal.

2. We catalog and demonstrate several novel use cases that Iris enables and host the dapps developed at https://apps.idealabs.network

3. We intend to consult with UX professionals to ensure that work developed as part of this is consistent, aesthetically pleasing, and approachable to users.

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide both **inline documentation** of the code and a basic **tutorial** that explains how a user can (for example) spin up one of our Substrate nodes and send test transactions, which will show how the new functionality works.
0c.	Testing Guide	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests. We will provide a demo video and a manual testing guide, including environment setup instructions.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
		We will publish a medium article explaining what

0e.	Article	was achieved as part of the grant, as well as any additional learnings that we deem important throughout development. https://medium.com/ideal-labs
1.	Javascript SDK	We will develop the javascript SDK to provide easy accessbility for front end developers to hook into dapps built on top of Iris as well as easily accomplish data ingestion, ejection, allocation/inclusion to data spaces, and more. We will document the full functionalities and specification of the SDK as part of this milestone.
2.	Examples	We develop several small applications using Iris to showcase potential capabilities, such as an NFT marketplace, online bookstore, or spotify-like application. If possible, we will also explore integration into an existing open source application.
3.	Hosting and connection the testnet	We host demo applications and capabilties using the idealabs.network domain by building a hub to access them. Further, we run these applications on our testnet which is deployed as part of milestone 3.

Future Plans

There are several key features of Iris that are out of scope of this proposal that the team will implement after these milestones. These items are specifically:

- The implementation of an anonymous repuation and feedback system
- The implementation of moderation protocols within dataspaces
- The implementation of machine learning algorithms, bayseian filters, and other checks to verify data integrity and compliance, both globally and within the context of data spaces.
- The implementation of governance protocols, including governance as a result of moderator node actions (with the implication being that Iris becomes an opinionated network where data may be rejected and specific addresses blocked from participation in the network).

Other non-development activities that we will pursue also include:

- We will finalize the Iris whitepaper.
- We will work with dapp developers to assist in develop of applications on Iris. We are already in communication with several projects that are interested in leveraging Iris as their storage system.
- We will strive to become a parachain on Kusama and Polkadot
- We will continue to expand the features available in Iris, such as partial ownership of assets, selling ownership of asset classes, and more

Additional Information :heavy_plus_sign:

How did you hear about the Grants Program? We first heard about the grants program from the Web3 Foundation Website.

Previous work on Iris is known the W3F grants team.